

Union Agreements: Entry, Editing & Structure

Status: Draft v1 · **Date:** July 10, 2026 · **Owner:** Ben · **Companion doc:** the [visual walkthrough](#) has a screen-by-screen mockup for every UI change described here, with the matching database notes beside each one. This PRD is the narrative and the spec; the walkthrough is the picture book. Read them together.

CONTENTS

1. The problem: real contracts don't fit, routine updates need engineers, and failures are silent
2. Five concepts to understand first (read before anything else)
3. The changes, by area — what and why
4. Database changes — before and after
5. What we are deliberately NOT doing
6. Migration notes & how we'll know it works

1 · The problem: real contracts don't fit, routine updates need engineers, and failures are silent

Scope of this PRD: the union agreement system — the setup wizard, editing of published agreements, the rate lookup that calculators and bids read, and the database tables underneath all of it.

Union agreement data is the cost foundation of the whole platform. The Rate Builder prices crews from it, bid estimates are built on it, and payroll will eventually pay people from it. If a wage or benefit number is wrong, everything downstream is wrong — quietly.

The agreement wizard in production today works well for creating a straightforward agreement. But we reviewed it page by page against a stack of real signed contracts (locals 53, 66, 70, 876, 1002, 1186, the California and Tennessee multi-local agreements, and others), and three kinds of gaps showed up:

- **Real contract structures we can't represent.** Some contracts have several wage tables under one agreement. Some print two sets of wages — one per state. Some benefit funds change rates on dates that have nothing to do with the wage schedule. Today, none of those fit.
- **Routine events with no path.** The single most common thing that happens to an agreement — the union announces next year's rates — cannot be entered without engineering help. Neither can an

extension, and "what happens when an agreement expires" has the wrong answer today.

- **Silent-failure risks.** Saving a wage edit that omits a classification silently deletes it. A rate lookup can return a rate from the wrong time window. New rows pre-fill with fake names ("New class") that pass validation. These aren't hypothetical — the GRI-1337 hotfix was one of this family.

The goal, in one sentence: a Gridbase admin can create any agreement in our contract stack, keep it current for years — raises, extensions, renewals, corrections — entirely through the UI, and every number stays trustworthy enough to eventually run payroll on.

2 · Five concepts to understand first

These are new ideas (or corrected ideas) that the rest of the document depends on. Each one exists because a real contract forced it.

Concept 1 — An agreement's expiration date is an alarm clock, not a kill switch

Every contract prints an end date. But almost every contract also says it *keeps going* past that date. From Local 1186's agreement, and nearly word-for-word in seven others we checked:

"...shall remain in effect until Midnight August 22, 2026... It shall continue in effect from year to year thereafter... unless changed or terminated in the way later provided herein."

So when the printed date passes, the crews are still working and still must be paid at the last published rates — that's what the contract itself says. Today our system treats the expiration date as the end of the data. After this PRD:

- The expiration date stays required — it drives the "go ask the union for the renewal" alerts, which is its real job.
- When the date passes, **rates keep resolving**, and the agreement displays an honest status: *"Using last known rates — term ended 8/28/27, renewal not on file. Confirm with the union."* This status is **computed whenever it's displayed** (is it published? is today past the end date? is there no successor?) — it is never stored, so nothing has to run on a schedule to flip labels, and it can never be stale.
- Rates only stop when a **successor agreement replaces this one** (a checkbox at publish time) or a person **deliberately marks the agreement ended**. The calendar alone never stops rates.

Concept 2 — Rate books: one agreement can contain several classification lists

Local 1002 has one agreement, one term, one benefits schedule — and **separate wage tables** for Outside Line work, Transmission Technicians, and Substation Technicians. Each table has its own job classifications and its own top rate that the others are calculated from.

We're calling each of those tables a **rate book**. An agreement contains one or more rate books; each book has its own classifications and its own anchor (the 100% rate the others derive from). The same classification name may appear in two books at different pay — that's real and legal.

The rule for when to use books vs. separate agreements: same signed document, same term, same benefits schedule → one agreement with rate books. Separate documents or separate terms (like Local 104's North and South agreements) → separate agreements.

Most agreements have exactly one book, and for them nothing changes visually — the book layer is invisible until someone adds a second one.

Concept 3 — Wage columns: the same table can carry more than one set of dollar values

Local 70's wage sheet prints one list of classifications with **two money columns: Maryland and Virginia**. The structure is identical — General Foreman is 115% of the Journeyman rate in both states — only the dollars differ. The whole two-column page is generated by the classification ladder plus two anchor values (\$45.87 MD, \$44.44 VA — and 115% of those gives exactly the printed \$52.75 and \$51.11).

We're calling each money column a **wage column**. Columns aren't only geographic: Local 1002's Outside Line book prints Standard and Federal Projects values — same idea. Entering a second column means typing one anchor and a handful of flat rates; every derived row computes itself.

Benefits and role-mapping are **not** affected by columns — fund rates are shared, and a classification maps to an internal role once. Which column applies is decided later, by the job (its state, its project type).

The full hierarchy, with every level optional-by-default:

```
Agreement  (LU 1002 – Outside Line & Industrial/Commercial, 8/17/25–6/30/29)
└─ Rate book  (Outside Line · Transmission Tech · Substation Tech)
    └─ Wage column  (Standard · Federal Projects)
        └─ Classification × Wage period  (Journeyman Lineman × 8/17/25–6/27/26)
```

Anything picking rates (the Rate Builder, bids) walks that path: **Local → Agreement → Book → Column → date** — and any level with exactly one option auto-selects and hides, so simple agreements feel exactly like today.

A storage detail that is easy to misread as a contradiction: when an agreement covers several locals (California's covers two, Tennessee's seven), the database stores one synchronized copy of each classification *per covered local*. This is not something anyone manages — the admin always edits **one** classification list and types each anchor **once**; the backend fans the identical rows out to every covered local in the same save (this fan-out already exists in production — the publish and edit code keeps per-local rows "in lockstep" automatically). The copies exist purely because everything that consumes rates today looks them up by (local, classification), and keeping that shape means no consumer rewrite. Per-local rows never diverge within an agreement: pay that differs by geography is what **wage columns** are for.

Concept 4 — A value is Set, TBD, or Not applicable — never ambiguously blank

Today a wage or benefit cell is either a number or a `tbd` flag, and underneath, an empty value and a zero look alike. That ambiguity is a bug factory — the GRI-1337 hotfix (benefit dropdown showing dates with no usable values) was exactly this.

After this PRD, every stored value carries one explicit status:

- **Set** — a real number from the contract.
- **TBD** — the union scheduled this period but hasn't announced the amount yet. In effect, unknown.
- **Not applicable** — deliberately no value (this fund doesn't apply here). Finished business, not missing data.

One deliberate exclusion: contracts sometimes say things like *"up to an additional \$0.50 if needed."* We do **not** store that as a special "contingent" value. We store the amount actually in effect, and if the increase is ever invoked, it gets entered on that day as an ordinary dated rate change. The system records facts, not possibilities. (The contract's wording can live in a note.)

What consumers do when they hit a TBD value — one rule, owned by the rate lookup: *the lookup never invents a number and never substitutes one silently. When the value covering the requested date isn't Set, it returns the newest Set value before that date, together with a freshness label; when no Set value has ever existed, it says so explicitly.*

Concretely: pricing a crew six months out where a classification's period is TBD returns that classification's **most recent Set rate**, plus metadata — the date it was set and a "TBD — priced at last known rate" flag. Every consumer must surface that flag (a footnote in the Rate Builder, a line-item warning on a bid: *"3 classifications priced at rates last set 9/1/25; the 9/1/26 raise is announced but not yet allocated"*). The estimator can still plan, knows exactly which numbers are soft, and knows to expect a retroactive true-up — which is how this industry already handles late-allocated raises. A **Not applicable** value contributes nothing and warns about nothing. A classification with **no Set value ever** is genuinely unpriceable: excluded with an explicit "no usable rate" warning on that

line only — **never \$0.00**, and never a hard error that blocks the rest of the calculation. This matches the instinct already in production (the GRI-1337 fix hides all-TBD dates rather than pricing them at zero) — stated here as the single rule so no consumer team has to guess.

Concept 5 — Every edit is plain, and every edit is logged

After an agreement is published, people fix typos, enter newly announced rates, add classifications, replace documents. Two principles govern all of it:

- **Editing is never a ceremony.** Click the cell, type, save. No approval flow, no locking, no "amendment" ritual for fixing \$54.44 to \$54.40.
- **The log is a side effect the user never performs.** Every change — from every code path — writes a history entry: who, what, before → after, when, from which screen. Entries are field-level and readable ("Ben changed Journeyman Lineman ST (Outside Line · Maryland · period 9/1/25–8/31/26) from \$57.60 to \$57.65"), never edited or deleted, and shown on a History tab. Because every entry has before-and-after values, "what did this agreement say on June 1?" is always answerable — which is what makes no-locking safe, and what payroll audits will demand.

The catch that makes this a real work item: today, Django admin and management scripts can write to these tables without leaving any trace. An audit trail with side doors isn't one. The invariant is **"no unlogged writes"** — not "no admin writes": admin becomes read-only on the financial tables by default and scripts route through the same service layer as the UI, but a **break-glass engineering role** keeps emergency admin write access — mandatory reason, every change auto-logged into the same edit history tagged "via break-glass admin," and an alert fired on every use. An emergency exit differs from a side door in exactly one way: it's wired to the alarm. (Banning it entirely wouldn't remove 2 AM emergencies — it would push them into raw SQL, the least logged path of all.)

3 · The changes, by area

Each change below says what it is and why. The [visual walkthrough](#) has a drawn mockup for every one of these — look for the matching "Proposed" frames on each step.

A · Setup step

A1 Documents become "upload what arrived, tag what it contains." Today there are two fixed upload slots (Union Contract, Wages & Benefits sheet) plus a "combined file" toggle. Real paperwork doesn't fit: sometimes the contract itself defines the benefits and the wage sheet is separate; Local 1002 sends a standalone fringe sheet plus one wage sheet *per rate book*. New model: a single file list.

Upload each PDF; tag it with what it contains using a per-file multi-select (closed state always shows the chosen tags as chips — "Contract · Benefits ·"). Options: Contract, Wages, Benefits (any combination), or Supporting document (exclusive — picking it clears the others). A fresh upload is amber "needs tagging" and **blocks Continue until someone says what it is — silence never classifies a file**. The gate changes from "are the slots filled" to "is each of Contract / Wages / Benefits covered by at least one file, and is nothing untagged," shown as a coverage bar. Empty state: the upload dropzone is the section; once files exist it collapses to an "+ Add document" row that stays a drop target.

A2 Expiration semantics per Concept 1. The final wage period may have an open end ("until amended"). Past the expiration date the agreement shows the computed "last known rates" status. Two new lifecycle outcomes exist: **Superseded** (set by a checkbox when publishing the replacement — see F1) and **Ended** (an explicit admin action with a required reason, for true terminations). Expiration alerts continue working unchanged.

B · Classifications step

B1 Rate books (Concept 2). An "+ Add rate book" affordance; with more than one book the table groups under book header rows, each book holding its own classes and anchor. Book names are free text, mirroring the printed sheet titles. Derivations ("110% of...") can only reference classes in the same book. One book (the default) renders exactly today's UI.

B2 Kill the "New class" pre-fill. Adding a row currently seeds the literal text "New class," which the user must delete — and which passes the name-required validation, so it can be published as real data. New rows start empty with a gray placeholder ("e.g. Journeyman Lineman"), autofocused; a blank row blocks Continue via the existing required-name rule. Identical fix for "New fund" on the Benefits step.

C · Wages step

C1 Move the OT/DT multipliers above the matrix, and make their effect visible. Today two small inputs sit in a footer bar under a wide scrolling table — easy to never see, though they multiply every rate in the agreement. They become a titled section *above* the matrix ("Overtime & double time — applied to every straight-time rate below") with a live example line: "Journeyman Lineman P1: \$53.41 ST → \$80.12 OT → \$106.82 DT," recomputing as you type. Why they're editable at all: California's agreement has **no time-and-a-half tier** — "double the straight rate shall be the maximum compensation for any hour worked" — so its OT multiplier must be 2.0. Guard: values outside the expected set ($OT \notin \{1.5, 2.0\}$ or $DT \neq 2.0$) get a soft inline "unusual — double-check the contract" note. The Review step's currently-permanent "confirm multipliers" warning then fires **only when the defaults were never touched**.

C2 Wage columns (Concept 3). A pill selector above the matrix (Maryland | Virginia | + Add wage column). Switching pills shows that column's values; derived rows recompute from that column's anchor. The live example in C1 uses the selected column's anchor.

C3 Value status (Concept 4) replaces the `tbd` flag on wage cells: Set / TBD / Not applicable.

D · Benefits step

D1 Two sections: funds that follow the wage periods, and funds with their own change dates.

Local 53's health & welfare fund changes every January 1 while its wages change on the September anniversary; Local 1002's fringe sheet has a change date matching no wage period at all. Funds that follow the wage schedule stay in today's period grid, untouched. Funds that don't get their own section: a simple list of dated changes (date + rate + optional note), entered exactly as the contract prints them — each rate applies until the next row, no end dates to manage. When adding a fund, one plain question routes it: "When does this fund's rate change? With the wage periods / On its own dates."

D2 The Basis picker: two presets and a Custom panel — one model. Today Basis is a two-value dropdown (\$/hr or % gross), which can't express how many funds are actually calculated. The new dropdown keeps two presets — **"\$ per hour worked"** (every hour actually worked, counted once — the standard) and **"% of gross pay"** — which are *saved answers* to three questions, plus **Custom...**, which opens those questions in a slide-over, pre-filled from the current pick: **(1)** what the amount multiplies (\$ per hour / % of pay / flat amount); **(2)** which hours count (only hours actually worked / all paid hours incl. PTO & sick / a single hour type); **(3)** what rate OT and DT hours get (same rate / the agreement's own premium multipliers from the Wages step — the label must interpolate the real values, never hardcode "1.5" / custom printed \$ rates, e.g. "\$4.00 ST, \$5.00 OT"). % funds get three independent base flags (all vs worked-only hours; include benefit earnings?; count OT at straight-time?) plus an optional "plus \$ per hour worked" add-on ("5% of gross plus \$1.25/hr" is real contract language). Flat funds: cadence (day/week/month/pay run) + trigger (any hours vs only when hours were worked). An **"Other"** option stores unusual wording verbatim and flags the fund for review — never force a wrong approximation. A "check with a real week" box (40 Reg + 10 OT → the computed dollars) recalculates live on every change — it's a component, not copy. This model was adapted from HCM TradeSeal's production calculation methods and survived an external adversarial review; prevailing-wage/401k/annualization calculations were deliberately excluded as payroll-engine concerns, not agreement facts.

D3 Fund names become plain free text. The typeahead picker (and its hardcoded fallback fund list) goes away. Reason: a suggested "close match" risks silently merging two genuinely different funds — an invisible, payroll-corrupting error — while duplicates from formatting differences are visible and fixable later. Enter the fund exactly as the contract prints it; the only check is no duplicate names within the same agreement. Linking entry names to canonical funds ("is this A&T fund the same trust as that one?") becomes deliberate back-office curation, later.

D4 Rename the "Pays" column to "Paid by", values spelled out: Employer / Employee (tooltip: Employer = paid on top of wages; Employee = deducted from the worker's pay). No ER/EE abbreviations.

D5 Funds can be scoped to a rate book. Some agreements contain trades with legally separate fringe obligations — a fund that applies to one book's classes and must never touch another's. The data model already handles this, because **benefit values are stored per classification** (and every

classification belongs to a book) with Not applicable covering exclusions. What's missing is the entry convenience: the fund's "Applies to" setting gains a middle grain — **every class / all classes in one rate book / specific classes**. Choosing a book fills that book's classes and marks every other book's classes Not applicable, in one act. All three grains compile down to the same per-class value rows, so the rate lookup never has to care which grain was used. (Real example: LU 1002's NEAP annuity is printed "See Classification," with different amounts across its books — per-class values are how the corpus actually works.)

E • Mapping step

E1 Remove the bulk "Auto-suggest mappings" button; keep the per-row hints. The blue "↳ Use 'Groundman / Driver'" link under an unmapped row stays — each acceptance is an individual human click, which guarantees row-by-row review. Bulk-apply is how a plausible-but-wrong match lands at scale.

E2 Group by rate book when the agreement has more than one — book header rows with per-book progress ("Outside Line · 11 of 13 mapped") inside one continuous table; combined count in the footer. The same class name in two books maps independently, without ambiguity — consumers always know which book they're in.

E3 "Complete" means zero unmapped, period. A class marked Not Available counts as handled — the badge answers "do I need to act?" and nothing else. Two rules ride along: Not Available is **always a human choice** (no script may ever auto-assign it — a legacy script did exactly that, which is how old dashboards lied), and detail views may show the three counts (mapped / not available / unmapped) for anyone who drills in.

E4 Retire the Settings → Union Mapping admin page and its API. Mapping is edited in exactly two places, both on the agreement: this wizard step at creation, and the agreement's detail page after publish. Bundled fix: the mapping save endpoint currently deletes a mapping when a row arrives with an empty id — a full payload can wipe untouched rows. That behavior ends. The replacement contract, precisely: **the payload is a list of explicit per-row instructions, and only touched rows are sent**. Each instruction carries a verb — *set* (with the internal role id), *mark not available*, or *clear* (return the class to unmapped). A row absent from the payload is never modified, and *clear* can only be produced by the user's explicit "Clear selection" action on that row (which already exists in the dropdown) — a bare null can never arrive by accident of serialization. No separate delete endpoint and no deleted-flag: a mapping is a **state of the classification**, not an entity, so clearing it is a light, visible, reversible state change — unlike removing a classification or fund, which stays a confirmed explicit action per G3.

F • Review & Publish step

F1 Make the "already-covered locals" notice real and actionable. It currently reads from a hardcoded mock file; point it at the API. And since a local legitimately holds several active agreements (Local 70 runs two at once), the notice stays an advisory — but gains a checkbox: *"This replaces [prior*

agreement] — *mark it superseded.*" Publishing a renewal then archives its predecessor in the same act, and the replaced agreement records which agreement superseded it.

F2 Previews handle the new structures. The benefit preview's "P1" cell shows the value in effect on the first day of period 1 — so an own-dates fund previews as a real number, never a blank that looks like a save failure. With multiple books/columns, the checklist and summary count them ("2 rate books · 2 wage columns") and the wage preview groups by book, columns side by side.

G · After publish: editing on the agreement's detail page

G1 Plain inline edits on the detail page's Wages / Benefits / Mapping / Documents tabs — click, type, save, logged (Concept 5). The agreement's name and expiration date are directly editable on the Details tab.

G2 "Add raise date" — the flagship new flow. The wizard is the only place wage periods can be created today, and it only works on drafts — so the single most common event (the union announces new rates) requires engineers. New action on the published agreement: enter a raise date. If it's after the schedule ends, a period is appended (extensions, holdover raises). If it falls *inside* an existing period, that period splits — e.g. year 4 of a 5-year deal splits into two half-year rates; the first half keeps its values unchanged, the second half opens for editing seeded from the first. **Invariant: answers for any date before the new raise date never change.** If the date is past the expiration, the same form requires the new expiration date, typed from the paperwork — never inferred; an agreement must never contain a period outside its own term. Rate entry groups by book and carries each wage column, like the Wages step.

G3 Deleting is always explicit. Today, saving wages with a classification missing from the payload silently soft-deletes it. That ends: removing a classification, fund, rate book, wage column, or covered local is a deliberate, confirmed action — never a side effect of saving.

G4 Covered locals are editable post-publish: adding a local fans out rate rows for it; removing one is its own confirmed action (it ends coverage — not a field tweak).

G5 "Mark agreement ended..." — admin-only, requires a typed reason, for the rare true termination with no successor. This and the F1 supersede checkbox are the *only* ways rates stop resolving.

G6 The edit history (Concept 5), concretely: rename the existing "amendments" concept to **edit history**; entries become field-level readable sentences naming the rate book and wage column when the agreement has them; coverage extends to every flow in this PRD; entries are append-only; and the side doors close under the "no unlogged writes" invariant: admin read-only on financial tables by default, scripts through the service layer, and a break-glass engineering role for emergencies (mandatory reason · auto-logged with a "via break-glass admin" source tag · alert on every use). The bones exist — the backend already snapshots before/after with a version counter and blocks concurrent overwrites.

H · Consumers (Rate Builder, bids)

H1 The picker follows the hierarchy — Local → Agreement → Rate book → Wage column → effective date — with single-option levels auto-selected and hidden. The benefits-package pick asks no book or column question — not because funds are agreement-wide, but because **benefit values are stored per classification** (see D5), so once the classifications are chosen, the right fund values follow automatically. Pay/benefits mix-and-match works unchanged. Breadcrumbs display whichever levels were real.

H2 One date-window rule, everywhere. The GRI-1337 hotfix taught us the canonical rate lookup ignores a rate row's end date, and the fix was applied at five call sites while ~14 others still use the old logic — so the same date can resolve to different rates depending on code path. Centralize the corrected logic in the model-level lookup (`get_current_rate`) and route every caller through it. An "expired" final period with an open end keeps resolving (Concept 1); everything else respects its window.

H3 The lookup's return contract covers uncertainty, not just numbers. Per the TBD rule in Concept 4: the centralized lookup returns (*value, the date that value was set, freshness flag*) — where the flag distinguishes a clean current value, a value carried forward past a TBD period ("priced at last known rate"), and a value from an agreement past its term ("using last known rates" — Concept 1). Consumers render the flag; they never decide fallback behavior themselves. A classification with no Set value ever returns an explicit no-rate result that consumers exclude with a visible per-line warning — never \$0.00, never a calculation-wide failure. **The flag is structured data, not a display string** — UI labels like "using last known rates" are derived from it, so headless and automated consumers get the identical contract: any API endpoint that serializes a rate carries the same metadata fields, and consuming rates by reading the tables directly (bypassing the lookup) is prohibited for the same reason the write side doors close in Concept 5 — a number stripped of its status metadata is not a rate.

4 · Database changes — before and after

Grouped by table. "Before" is production today. Every change traces to a lettered item above.

UnionWageAgreement — the agreement container

FIELD	BEFORE	AFTER	WHY
lifecycle_status	DRAFT / PUBLISHED / ARCHIVED	adds SUPERSEDED and ENDED	F1, G5 — the two deliberate ways an agreement stops governing. "Last known rates" is not a status value: it's computed at display time (published + past end date + no successor), so it can never go stale.

FIELD	BEFORE	AFTER	WHY
superseded_by	—	new — FK to the replacement agreement	F1 — renewal chains become traceable.
ended_on	—	new — date of explicit termination	G5 — the reason text lives in the edit history entry.
is_active	a second, independent on/off switch	deprecated — derived from lifecycle_status during transition, then dropped	Two sources of truth for "is this live" will eventually disagree. One switch.

AgreementPeriod — the wage-period spine

FIELD / RULE	BEFORE	AFTER	WHY
end_date	required on every period	nullable — but only the last period may be open	A2 / Concept 1 — the final period runs "until amended," matching the contracts' own year-to-year language.
row creation	only the wizard's setup step, only on drafts (full delete-and-recreate)	"Add raise date" (G2) inserts and splits periods on published agreements, resequencing safely	The annual raise, extensions, and mid-term splits become self-service.

New table: RateBook · New table: WageColumn

TABLE	FIELDS	WHY
RateBook	agreement FK · name (free text) · sort order	Concept 2 / B1. Every agreement gets one default book on creation (and existing published agreements get one backfilled), so single-book agreements behave exactly as today.
WageColumn	rate_book FK · label (free text, e.g. "Maryland", "Federal Projects") · sort order	Concept 3 / C2. Same default-and-backfill approach: one column per book unless the contract has more.

UnionMappingOption — one row per classification per covered local

FIELD / RULE	BEFORE	AFTER	WHY
rate_book	—	new FK — every classification belongs to one book	B1.

FIELD / RULE	BEFORE	AFTER	WHY
uniqueness	(name, local, agreement)	(name, book , local, agreement)	The same class name legally recurs across books at different pay (LU 1002).
derivation base	any class in the agreement	must be in the same book	Each book has its own anchor; "110% of JL" must be unambiguous.
one row per covered local	production behavior today: the classification list is fanned out to one synchronized row per covered local ("lockstep" updates in the publish/edit code)	unchanged — and now a stated invariant: sibling rows are identical by construction; only the single write path may touch them; admins always edit one logical list	Compatibility: every consumer looks rates up by (local, classification). Geographic pay variance is modeled by wage columns, never by diverging per-local copies — so entering the Virginia anchor once still populates every covered local automatically.

UnionMappingOptionRate — the wage rate rows

FIELD / RULE	BEFORE	AFTER	WHY
period	— (start/end dates are copies of the period's dates, with no link back)	new FK to AgreementPeriod; the date columns become derived from it	Copied dates drift when periods change; and G2's period-splitting only works if rate rows follow their period. This is the prerequisite for "Add raise date."
wage_column	—	new FK — one rate row per class × period × column	C2. Single-column agreements have exactly as many rows as today.
value_status	tbd boolean; empty vs zero ambiguous	SET / TBD / NOT_APPLICABLE enum replaces it	Concept 4 — retires the GRI-1337 bug class at the source.
lookup rule	get_current_rate ignores end dates; 5 call sites patched separately (GRI-1337), ~14 unpatched	one corrected model-level lookup; every caller routed through it	H2 — the same date must never resolve to different rates by code path.
uniqueness	none — duplicate "active" rows possible; correctness rests on an is_active flag toggled in serializer code	database constraint: one active row per (class, period, column)	Rules in the database survive every write path, including ones not written yet.

UnionMappingOptPackageRate — the benefit rate rows

FIELD / RULE	BEFORE	AFTER	WHY
effective dates	copies of wage-period dates	own authoritative date windows — a fund's schedule is independent of the wage periods when it needs to be	D1 — Local 53's January fund changes vs September wage changes.
value_status	tbd boolean	SET / TBD / NOT_APPLICABLE (same enum as wages; no "contingent" — see Concept 4)	One vocabulary for every value in the system.
linkage (no change — stated invariant)	each value row links to a specific classification (fund × class × period)	unchanged — and since every classification now belongs to a rate book, fund values are book-addressable through their class; Not applicable expresses "this fund never touches that book/class"	D5 / H1 — mixed-trade agreements (fringes legal for one book, illegal for another) model without splitting the agreement. Book-level scoping in the UI is an entry shortcut that compiles to these same rows.

Fund configuration (today's UnionPackages.rate_type) — the basis fields

FIELD	BEFORE	AFTER
rate_type	one 5-value list (PERCENTAGE / PER_HOUR / PER_DAY / PER_MEAL / FLAT_MONTHLY)	replaced by decomposed fields below; each legacy value maps 1:1 onto a default combination (e.g. PER_HOUR → hours · worked-only · same rate)
base_type	—	HOURS / PAY / FLAT
hours_scope · ot_dt_treatment	—	worked-only / all-paid / single-type × SAME / AGREEMENT_PREMIUM / CUSTOM

FIELD	BEFORE	AFTER
ot_rate_override · dt_rate_override	—	explicit \$ rates when treatment = CUSTOM
pay_hours · pay_incl_benefits · pay_ot_as_straight	—	three independent flags for % funds
addon_per_hour	—	optional flat \$ per worked hour on a % fund
flat_cadence · flat_trigger	—	DAILY / WEEKLY / MONTHLY / PER_PAY × any-hours / worked-hours
basis_other_text	—	verbatim contract wording + needs-review flag
applies_scope · applies_book	applies_to: ALL / CLASS_SPECIFIC	EVERY_CLASS / RATE_BOOK (+ book FK) / SPECIFIC_CLASSES
shape rule	—	database check: only the chosen base_type's fields may be set

AgreementDocument — uploaded files

FIELD / RULE	BEFORE	AFTER	WHY
document_type	one value from CONTRACT / WAGES_BENEFITS / CONTRACT_AND_WAGES / OTHER — 3 of the 7 real combinations	replaced by four flags: covers_contract · covers_wages · covers_benefits · is_supporting. Legacy values map 1:1.	A1 — a named-combination list always lags reality (a contract-that-defines-benefits had no value). Flags express all combinations, forever. All four false = "needs tagging" (unfinished), so silence never classifies a file.
one-file-per-type rule	enforced in the serializer	removed	Multiple wage sheets are legal — LU 1002 has one per rate book.

AgreementAmendment — becomes the edit history

ASPECT	BEFORE	AFTER	WHY
concept & name	"Amendments," section-level JSON snapshots on V2 edits only	Edit history: field-level entries (who, what, before → after, when, source screen; book & column named when present), covering every flow in this PRD, append-only	G6 / Concept 5. "Amendment" is a loaded contract term we may need for real MOUs someday; this table is a change log.
coverage	Django admin and management scripts write tables directly, unlogged	admin read-only by default; scripts call the service layer; a break-glass engineering role retains emergency admin write — mandatory reason, auto-logged with a distinct source tag, alert on use	The invariant is "no unlogged writes," not "no admin writes" — a total ban would push 2 AM emergencies into raw SQL, the least logged path of all.

5 · What we are deliberately NOT doing

Each of these was considered and consciously deferred or rejected — so nobody re-litigates them mid-build.

- **No locking or approval workflows.** Edits apply immediately; the edit history is the safety net. Locking tiers (e.g. freezing data used by a submitted bid) come later, when bids and payroll create real consumption to protect.
- **No "contingent" value status.** See Concept 4 — facts, not possibilities.

- **No standard classification taxonomy and no fund catalog codes.** Class and fund names mirror the printed page, deliberately. Cross-agreement canonical linking is future back-office curation.
- **No overtime *trigger* rules** (daily-8 vs weekly-40 etc.) — calculators and bids only need the multipliers; triggers become necessary with payroll and get their own design then.
- **No prevailing-wage / 401k-adjustment / annualization calculation types** — those are payroll-engine and job-context calculations, not facts printed in a collective bargaining agreement.
- **No changes to how per diems are entered**, beyond funds gaining honest value statuses.

6 · Migration notes & how we'll know it works

Migration in one pass, no data re-entry

- Every existing agreement gets **one default RateBook**, and every book **one default WageColumn**; all existing classifications and rate rows attach to them. Single-book, single-column agreements are pixel-identical to today.
- Rate rows gain their **period FK** by matching the dates they already carry; the date columns then become derived.
- `tbd=true` → status TBD; a real value → SET; the NULL/0 rows GRI-1337 fought get resolved to NOT_APPLICABLE or TBD during the migration (this is the one place human review of flagged rows may be needed).
- Legacy `rate_type` and `document_type` values map 1:1 onto the new fields as tabled above.
- A one-time constraint-preparation pass dedupes any conflicting "active" rate rows before the uniqueness constraints land.

Acceptance — five scenarios that must work end to end

1. **Enter LU 1002 completely** — one agreement, three rate books, two wage columns on the Outside Line book, the standalone fringe sheet's own-dates fund changes, documents tagged from the real PDFs — no engineering help.
2. **Enter LU 70's Virginia column** by typing one anchor and the few flat-rate classes; every derived row fills itself and matches the printed sheet ($\$44.44 \times 115\% = \51.11).
3. **The annual raise:** on a published agreement, add a raise date, enter the new rates, done — and a raise date past the expiration requires typing the new expiration in the same form.
4. **Let an agreement pass its expiration:** calculators keep pricing from it, clearly labeled "using last known rates"; publishing its successor with the supersede checkbox archives it.

5. **Audit any change:** every edit made in scenarios 1–4 appears in the History tab as a readable sentence with before → after values, including the book and column names.

Draft v1 · July 10, 2026 · Prepared from the page-by-page wizard review (Ben + Claude), the contract corpus (LU 53, 55, 66, 70, 876, 1002, 1186, CA, TN, MSECA), the HCM TradeSeal calculation-methods documentation, and the EPR Live technical guide. Visual companion: [the wizard walkthrough](#) — a mockup exists there for every UI change in section 3.